

箕面ハイキングマップ デプロイ手順書

バージョン: 1.1
最終更新日: 2026年8月19日
対象: 運用・開発担当者
関連:

機能仕様 `funcspec-202608.md` /
公開API 仕様 `publish-api-202608.md`

1. 本書について

本アプリの配信のしかたと、変更の種類ごとに何をすればユーザーに届くかをまとめる。

配信先が2つ（Vercel / GitHub Pages）あり、さらに地図データはデプロイを伴わない公開で更新されるため、「どれを直したときに何が必要か」を取り違えやすい。判断はここに一本化する。

2. 配信先と役割

配信先	配信するもの	反映のきっかけ
Vercel	アプリ（ <code>public/</code> ） + 公開API（ <code>api/</code> ） + 公開ストア（Vercel Blob）	<code>main</code> への push（リポジトリ連携）／環境変数の変更後は再デプロイ
GitHub Pages	アプリ（ <code>public/</code> ）のみ	<code>main</code> への push（ <code>.github/workflows/pages.yml</code> が自動実行）

- 公開APIは **Vercel** にしか無い。GitHub Pages 版アプリは `https://minoh-hiking.vercel.app/api/*` をクロスオリジンで参照する（`config.js` が `github.io` を判定して切り替える。API側はCORSを全許可）。
- したがって **API** を止めると両方のアプリでデータが出なくなる。

```
push (main)
|
+--> Vercel      : public/ (app) + api/ (publish API) + Blob (store)
|
+--> GitHub Pages : public/ (app) only
                        |
                        +-- GET --> https://minoh-hiking.vercel.app/api/*

MapPublisher -- POST /api/mapdata, /api/closures --> Blob -- GET --> 両方のアプリ
```

Vercel の Production Branch（既定は `main`）は Vercel の管理画面で確認できる。
本書は `main` を前提に書いている。

3. 変更の種類とやること（早見表）

変更したもの	必要な作業	デプロイ
地図データ・通行止めの内容	MapPublisher で公開する	不要
<code>public/</code> のコード・画像	<code>SHELL_CACHE</code> をバンプ → push (→ 5章)	要
<code>api/</code> のコード	push	要
Vercel の環境変数	値を設定 → 再デプロイ (設定だけでは反映されない)	要
<code>public/data/tile_manifest.json</code> (タイル範囲)	<code>SHELL_CACHE</code> をバンプ → push	要
<code>docs/</code> のみ	push (<code>.md</code> を直したら <code>.pdf</code> も作り直す)	影響なし

「データを直すたびにアプリを出し直す」必要は無い。ポイント・ルート・スポット・通行止めはすべて公開API 配信であり、MapPublisher からの公開だけでユーザーに届く。

4. 事前設定（初回・環境を作り直したときのみ）

Vercel プロジェクト **minoh-hiking** に必要なのは、**Blob ストアの接続と公開トークン**の2つだけ。
どちらも**設定した後の再デプロイで初めて有効**になる。4.1 から順に、1回だけ実施する。

画面の項目名は Vercel 側の更新で変わることがある。表記が違うときは
「Storage（ストレージ）」「Environment Variables（環境変数）」「Redeploy（再デプロイ）」
に当たる場所を探す。

4.1 Blob ストアを接続する

公開データ（地図データ・通行止め）の実体を置く場所。**接続するだけでよい**。
フォルダやファイルを手で作る必要は無く、初回公開時に
`api/_lib/store.js` が作る。

1. <https://vercel.com/> にログインし、プロジェクト **minoh-hiking** を開く
2. 上部タブの **Storage** を開く

3. **Create Database**（すでにあるストアを使うときは **Connect Store**）→ **Blob** を選ぶ
4. ストア名を入れて作成する（例: `minoh-hiking-blob`。名前は任意。Region は既定のままでよい）
5. 接続先プロジェクトに **minoh-hiking** を選び、Environment は **Production / Preview / Development** すべてにチェックして **Connect**
6. **Settings** → **Environment Variables** に `BLOB_READ_WRITE_TOKEN` が自動追加されたことを確認する（値は開かない・コピーしない・Git に入れない）

変数名は既定のままにする。接続時に Environment Variables のプレフィックスを付けて

`〇〇_READ_WRITE_TOKEN` にすると、`@vercel/blob` が読むのは `BLOB_READ_WRITE_TOKEN` だけなので公開時に `500` になる（`api/_lib/store.js` は token を渡していない）。

ストアの Settings → Quickstart の `.env.local` タブに出る `BLOB_STORE_ID` /

`BLOB_READ_WRITE_TOKEN` は「ローカル開発用にコピーする値」の表示で、そのままよい。

Vercel 上は接続で設定済み。`BLOB_STORE_ID` は本アプリでは使わない（OIDC 認証用）。

接続後、公開のたびに Blob 上へ次のパスが作られる（定義は `api/_lib/datasets.js`）。

Blob 上のパス	中身
<code>manifest.json</code>	各データセットの version・updatedAt・件数（ <code>GET /api/manifest</code> の実体、採番の基準）
<code>mapdata/minoh-hiking-mapdata.geojson</code>	緊急ポイント・ルート・スポット（最新）
<code>mapdata/previous.geojson</code>	同・前回分（1世代のみ）
<code>closures/minoh-hiking-closure.geojson</code>	通行止め・通行困難地点（最新）
<code>closures/previous.geojson</code>	同・前回分（1世代のみ）

フォルダを手で作る必要は無い。Blob に「フォルダ」という実体は無く、パス名の `/` を管理画面がフォルダのように見せているだけ。すでに運用している `closures/` はそのまま使い（本体のパスは旧方式から変えていない）、`mapdata/` と `manifest.json` は初回公開のときにできる。`previous.geojson` は退避元ができる2回目の公開から作られる。旧方式の `closures/history/` は新方式では使わない（削除は 6.3）。

ストアはプロジェクトではなく **アカウント（チーム）** に属する。作り直すと中身は空になり、`manifest.json` が無くなるため version の採番も 1 からやり直しになる。中身を残したまま別プロジェクトから使いたいときは、作り直さず **Connect Store** で接続する。

4.2 公開トークン `MAP_PUBLISH_TOKEN` を設定する

MapPublisher からの公開（POST）を通すための共有トークン。mapdata・closures 共通で1つ。

1. ランダムな32バイトの値を作る (PowerShell) :

```
$b = [byte[]]::new(32)
[System.Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
[Convert]::ToBase64String($b)
```

2. 環境変数の画面を開く (直接 URL が確実)

<https://vercel.com/<アカウント名またはチーム名>/minoh-hiking/settings/environment-variables>

画面からたどる場合は、プロジェクト **minoh-hiking** を開く → 上部タブ **Settings** → 左メニュー **Environment Variables**。

ここはプロジェクトの **Settings** で、Blob ストア側の Settings (Quickstart がある画面) には環境変数の追加欄は無い。

3. **Add New** (版により **Create new** / +) を押して登録する。入力欄が最初から出ている版もある。 **.env** を貼り付ける欄しか見当たらないときは **MAP_PUBLISH_TOKEN=値** の形式で貼ってもよい

項目	値
Key	MAP_PUBLISH_TOKEN
Value	1 で作った文字列 (32文字以上)
Environments	Production (必須)。Preview でも公開を試すなら Preview も
Sensitive	選べるならオン (登録後は値を読み出せなくなる)

4. **Save** する

5. 値はパスワードと同じ扱いにする。Git ・ **public/** ・ チャットに貼らない
(**.gitignore** が **.env** / **.env.local** を除外しているのは、ローカルに置いたときの保険)

4.3 再デプロイして有効化する

環境変数は、すでに動いているデプロイには入らない。設定したら必ず出し直す。

1. **Deployments** タブを開く
2. 最新の **Production** デプロイの右端 ... → **Redeploy**
3. ダイアログの **Redeploy** を押す (Build Cache の使用有無はどちらでもよい)
4. Status が **Ready** になるまで待つ

GitHub Pages 側には API が無いため、この作業は不要 (→ 2章)。

4.4 設定できたか確認する

```
# (a) トークン無しの POST。401 が正しい状態
curl.exe -s -o NUL -w "%{http_code}\n" -X POST -H "Content-Type: application/json"
-d "{}" https://minoh-hiking.vercel.app/api/mapdata

# (b) 配信状況。未公開なら version は空文字で返る
curl.exe -s https://minoh-hiking.vercel.app/api/manifest
```

結果	意味
(a) が 401	MAP_PUBLISH_TOKEN が有効。正しい状態
(a) が 503	トークン未設定、または設定後に再デプロイしていない (→ 4.2 / 4.3)
(b) が JSON を返す	API は動いている (Blob 未接続でも空の JSON が返るため、これだけでは接続の確認にならない)

Blob 接続の最終確認は、MapPublisher から実際に1回公開すること。

200 (version と count が返る) なら接続できている。

500 (公開ストアへの保存に失敗しました) なら Blob 未接続か、接続後に再デプロイしていない。

4.5 MapPublisher 側にトークンを登録する

公開操作を行う MapPublisher (別リポジトリの運用担当者用アプリ) に、4.2 と同じ値を保存する。

- Vercel 側だけ変えると公開が 401、MapPublisher 側だけ変えても 401 になる
- トークンを変えるときは、Vercel (+ 再デプロイ) と MapPublisher の両方を必ず揃える

4.6 完了チェックリスト

- [] Storage に Blob ストアがあり、minoh-hiking に接続されている
- [] BLOB_READ_WRITE_TOKEN が Environment Variables にある (自動追加)
- [] MAP_PUBLISH_TOKEN を Production に設定した
- [] 設定後に Redeploy し、Ready になった
- [] トークン無しの POST が 401 (503 ではない)
- [] MapPublisher に同じトークンを登録した
- [] 試しに1回公開して 200 が返った

ローカルで vercel dev を使って API まで動かすときだけ、Vercel CLI で環境変数を取り込む (vercel link → vercel env pull .env.local)。 public/ を静的配信するだけなら不要。

5. 通常のデプロイ（アプリの更新）

`public/` または `api/` を変更したときの手順。

データの内容だけを直したときは、この章は不要（MapPublisher からの公開で届く → 3章）。

5.1 手順

① 変更内容とブランチを確認する

```
git status --short
git branch --show-current    # main であること
```

② `public/` を変更したなら `SHELL_CACHE` をバンプする（`public/service-worker.js`）

```
Select-String -Path public/service-worker.js -Pattern "SHELL_CACHE = "
# 例: const SHELL_CACHE = 'app-shell-2026-08-19.1';
```

- 命名は `app-shell-yyyy-mm-dd.n`。日付は出す日、`n` はその日の連番（初回 `.1`、同じ日の2回目は `.2`）
- ⚠ 忘れると、端末に旧 UI と新 UI が混ざったまま残る。
アプリシェルは `stale-while-revalidate` で配るため、キャッシュ名が同じだと更新の確認（`SHELL_CACHE` 比較 → `confirm`）が働かない
- `api/` や `docs/` だけの更新ならバンプ不要

③ ファイルを追加したときは `SHELL_LOCAL_PATHS` にも足す（同じファイル）

- JS モジュール・アイコン・画像など、オフラインでも要るものはすべて
- 入れ忘れると、オフライン起動時にそのファイルだけ取得できない

④ 差し替えた旧ファイルは、このリリースでは消さない

- 端末には旧 `index.html` / 旧 `app.js` がキャッシュされたまま残ることがあり、消すと 404 になって画像が出ない・モジュールが読めない、といった壊れ方をする
- 削除待ちの一覧は `service-worker.js` の `SHELL_LOCAL_PATHS` 直下の注記にまとめてある
- 全端末が更新を通したと判断できる次のリリース以降に消す

⑤ ローカルで表示を確認する

```
cd public; python -m http.server 8123    # → http://localhost:8123/
```

- `/api/*` は 404 になる（ローカルに API は無い）。地図データ・通行止めが出ないのは想定どおりで、「バージョン情報」の該当行と件数は `-` になる
- API 込みで見たいときだけ `vercel dev`

⑥ コミットして push する

```
git add -A
git commit -m "変更内容の要約"
git push origin main
```

⑦ 2つの配信先が更新されたか見る

配信先	見る場所	正常
Vercel	Deployments タブ	当該コミットの Production が Ready
GitHub Pages	リポジトリの Actions → Deploy to GitHub Pages	緑（失敗なら <code>workflow_dispatch</code> で再実行）

⑧ 7章の動作確認を行う（Vercel 版・GitHub Pages 版の両方）

⑨ `docs/*.md` を直したときは `docs/*.pdf` も作り直す

5.2 ユーザーへの反映

- 起動時に「アプリの更新版があります」の確認が出る（「起動時にアプリの更新版を確認」が ON のとき）
- OK を押すと最新を取得して再読み込みする。**ダウンロード済みの地図タイルは消えない**
- 確認を出さない設定の端末でも、次回以降の読み込みで自然に最新化される
- 反映は**端末が次に起動したとき**。全端末に行き渡るまで日数がかかる前提で、旧ファイルの削除は次のリリース以降にする（→ ④）

6. 今回の移行デプロイ（2026.12・1回限り）

地図データをアプリ同梱から公開API 配信へ切り替えるリリース。
API とアプリを同時に出してはいけない。

6.1 なぜ順序が要るか

アプリ側を先に切り替えると、初回公開が済むまでの間 `GET /api/mapdata` が空を返し、
全ユーザーの地図からポイント・ルート・スポットが消える。

`main` への push は Vercel と GitHub Pages の両方を同時に更新するため、
2つのコミットに分け、間隔を空けて push する。

6.2 手順

#	作業	対象	確認
0	Vercel に <code>MAP_PUBLISH_TOKEN</code> を設定	Vercel	4章
1	コミットA （ <code>api/</code> + <code>docs/</code> + <code>README.md</code> ）を push	リポジトリ	<code>public/</code> は据え置き。ユーザーは同梱データのままだ動き続ける
2	MapPublisher で mapdata を初回公開	MapPublisher	<code>2026.1</code> が採番される

#	作業	対象	確認
3	配信内容を確認	curl	7.1
4	コミットB (<code>public/</code>) を push	リポジトリ	ここで初めてアプリが配信データを使う
5	表示を確認	ブラウザ	7.2

手順1の時点で `closures` の公開トークンは `MAP_PUBLISH_TOKEN` に変わる（旧 `CLOSURES_PUBLISH_TOKEN` は使われない）。手順0 を飛ばすと通行止めの公開が `503` になる。

6.3 リリース後の後始末（次のリリースで行う）

全端末がアプリ更新を通じたことを確認してから実施する。

#	作業	状態
1	<code>public/data/minoh-emergency-points.geojson</code> / <code>public/data/minoh-hiking-routes-spots.geojson</code> を削除	完了（2026-08-20）
2	<code>public/closures.js</code> を削除	未
3	<code>vercel.json</code> の <code>/data/(.*)</code> のキャッシュ設定を見直す（残るのは <code>tile_manifest.json</code> のみ）	未
4	Vercel の環境変数 <code>CLOSURES_PUBLISH_TOKEN</code> を削除	完了（2026-08-20）
5	Blob 上の旧履歴 <code>closures/history/</code> を一括削除（前回分1世代の方式に移行済みのため）	完了（2026-08-20）

手順1は全端末の更新完了を待たずに実施した。旧シェルをキャッシュしたままの端末が旧 `app.js` から取得を試みると 404 になるが、旧 `map.js` は `console.warn` して `return null` するだけで表示は継続するため、影響は緊急ポイント・ルート・スポットが表示されないことに留まる。アプリ更新を通せば公開API から取得するようになる。

手順2の `public/closures.js` は、旧 `index.html` / 旧 `app.js` が `import` しているため、削除するとモジュール読み込みが失敗して起動しない。こちらは全端末の更新完了を待つこと。

7. 動作確認

7.1 API 単体（curl）

Windows PowerShell では `curl` が別のコマンドの別名になっているため、`curl.exe` と書く。

```
# version と件数（数百バイト）
curl.exe -s https://minoh-hiking.vercel.app/api/manifest

# 本体（件数だけ数える）
curl.exe -s https://minoh-hiking.vercel.app/api/mapdata | node -e "let
s='';process.stdin.on('data',d=>s+=d).on('end',
()=>console.log(JSON.parse(s).features.length))"
curl.exe -s https://minoh-hiking.vercel.app/api/closures | node -e "let
s='';process.stdin.on('data',d=>s+=d).on('end',
()=>console.log(JSON.parse(s).features.length))"

# 認証：トークン無しの POST は 401 になること（401 以外なら公開口が無防備）
curl.exe -s -o NUL -w "%{http_code}\n" -X POST -H "Content-Type: application/json"
-d "{}" https://minoh-hiking.vercel.app/api/mapdata
```

確認すること:

- GET /api/manifest が mapdata / closures の version ・ updatedAt ・ 件数を返す
- GET /api/mapdata の件数が公開した件数と一致する（初回公開なら 668 件）
- 公開のたびに version の連番が1つ進む（2026.1 → 2026.2）
- トークン無し・誤トークンの POST が 401

7.2 アプリ側（ブラウザ）

Vercel 版 ・ GitHub Pages 版の両方で確認する。

- 「ハイキングマップ表示」で緊急ポイント・ルート・スポット・通行止めが描画される
- ルート線が端点（開始・終了ポイント）まで伸びている
- 「バージョン情報」に地図データ・通行止めの version と件数が出る（ - でない）
- 2回目に開いたとき、DevTools の Network に /api/mapdata が出ない
（version が一致するので本体を取りに行かない）
- 公開した直後に開き直すと、新しい version と件数に変わる
- オフライン（機内モード）で開いても、前回取得した内容が表示される

8. 復旧・ロールバック

状況	対処
誤ったデータを公開した	正しいデータをもう一度公開する（全置換）。直前の内容は Blob の previous.geojson に1世代だけ残っている
公開が 500 で失敗した	もう一度公開する。manifest.json が進んでいないため同じ version が採番される（幕等）。二重に version が飛ぶことはない
アプリの不具合を出してしまった	Vercel の Instant Rollback で前のデプロイに戻す。あわせて main を git revert する（戻さないと次の push で再発する）

状況	対処
GitHub Pages 側だけ古い	Actions の <code>Deploy to GitHub Pages</code> が失敗していないか確認し、 <code>workflow_dispatch</code> で再実行する

ロールバックしても、端末のキャッシュは `SHELL_CACHE` の名前で判断される。
戻した版のキャッシュ名が新しい版と同じだと更新が検知されないため、
切り戻し版でもキャッシュ名を1つ進めて出し直すのが確実。

9. トラブルシューティング

症状	主な原因	対処
公開が <code>503</code> (トークン未設定)	<code>MAP_PUBLISH_TOKEN</code> 未設定、または 設定後に再デプロイしていない	環境変数を確認して再デプロイ
公開が <code>401</code>	トークン不一致	MapPublisher の保存済みトークンを入れ直す
公開が <code>400</code>	データが検証に通らない	API が返す日本語メッセージのとおり直す (判定は <code>publish-api-202608.md</code> §6 にのみ置いている)
公開したのにアプリに出ない	端末がまだ起動し直していない	アプリを開き直す。公開は各端末が次に起動したときに反映される
旧 UI と新 UI が混ざる	<code>SHELL_CACHE</code> のバンプ漏れ	キャッシュ名を進めて出し直す
起動画面の画像やモジュールが 404	差し替えた旧ファイルを同じリリースで消した	ファイルを戻し、次のリリース以降に削除する
オフラインで地図データが出ない	Service Worker の掃除で <code>mapdata-cache</code> / <code>closures-cache</code> を消している	<code>service-worker.js</code> の <code>APP_MANAGED_CACHES</code> に入っているか確認する
GitHub Pages 版だけデータが出ない	Vercel 側の API が落ちている / CORS 設定の変更	<code>curl.exe</code> で Vercel の API を直接確認する

10. デプロイ前チェックリスト

- [] `public/` を変更した → `SHELL_CACHE` をバンプした
- [] JS ファイルを追加した → `SHELL_LOCAL_PATHS` に追加した
- [] 差し替えた旧ファイルを**このリリースでは消していない**
- [] 環境変数を変えた → 再デプロイする段取りになっている
- [] `docs/*.md` を直した → `docs/*.pdf` を作り直した
- [] データの内容だけの変更なら、**デプロイではなく MapPublisher からの公開**で足りると確認した

11. 変更履歴

版	日付	内容
1.0	2026-08-18	初版。配信先2つの役割、変更の種類ごとの作業、2026.12 の移行デプロイ手順（API とアプリを分けて出す）、確認・復旧・トラブルシューティングをまとめた
1.1	2026-08-19	4章（事前設定）と5章（通常のデプロイ）を画面操作・コマンドのレベルまで具体化。Blob ストアの接続手順・Blob 上のパス一覧・トークンの生成と登録・再デプロイ・設定確認（401/503 の読み分け）と、デプロイ手順の各ステップにコマンドと確認箇所を追記